



CELEBRATING 50 YEARS

Dilip Kumar Maripuri
Computer Applications







Data Structures

How is it Different?

Feature	Splay Tree	AVL Tree	Red-Black Tree
Balancing Mechanism	Adjusts dynamically using rotations (splaying)	Uses height-based balancing	Uses color-based balancing
Extra Data Stored	No extra balance information	Stores height balance factor	Stores color information
Performance Guarantee	Amortized $O(\log n)$, worst-case $O(n)$	Guaranteed $O(\log n)$	Guaranteed $O(\log n)$
Best Use Case	When frequently accessed elements need to be retrieved fast	When strict balancing is needed	When consistent balanced performance is required



Data Structures

Key Differences

► Balancing Approach:

- AVL and Red-Black Trees enforce strict balancing **on every insertion and deletion**.
- Splay Trees **do not guarantee strict balance**, but they adjust based on access patterns.

► Performance Trade-offs:

- AVL and Red-Black Trees always ensure $O(\log n)$ **worst-case complexity** for search, insert, and delete.
- Splay Trees may sometimes take $O(n)$ **in the worst case**, but a **sequence of operations remains $O(\log n)$ on average**.



Data Structures

Key Differences

► Best Use Cases:

- If **all elements are accessed equally**, AVL or Red-Black Trees are preferable.
- If **some elements are accessed more frequently than others**, a Splay Tree is a better choice as it moves those elements closer to the root.



Data Structures

Where is a Splay Tree Used?

► 1. Caches (Least Recently Used - LRU)

- Splay Trees are used in **LRU cache implementations** to store and manage recently accessed data.
- Since the most frequently accessed elements move to the root, they can be quickly retrieved.

► 2. Memory Management

- Operating systems use Splay Trees to manage **memory allocation** efficiently.
- Helps in dynamic memory allocation where frequently used blocks are quickly accessible.



To construct a Splay Tree, we perform three major operations:

1. **Splaying:** Moves the accessed node to the root using rotations.
2. **Insertion:** Inserts a new node and then splays it to the root.
3. **Search:** Searches for a node and then splays it to the root.

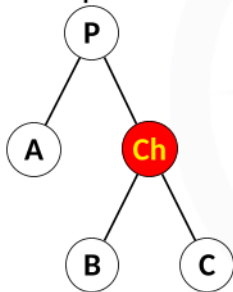


Data Structures

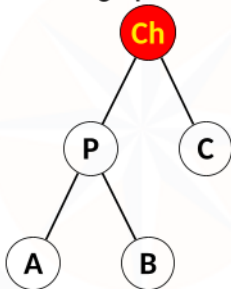
Splay Tree Operations

- ▶ In a **Splay Tree**, we perform **splaying** to move an accessed node to the root using rotations.
- ▶ The three main types of rotations used in splaying are:
 1. **Zig Rotation** (Single Rotation)
 2. **Zig-Zig Rotation** (Double Rotation in the same direction)
 3. **Zig-Zag Rotation** (Double Rotation in opposite directions)





After Zag Operation

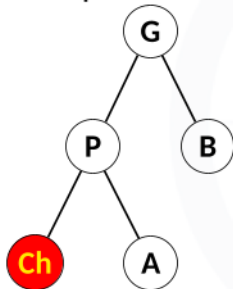


Occurs when the accessed node Ch is a Right child of the root.

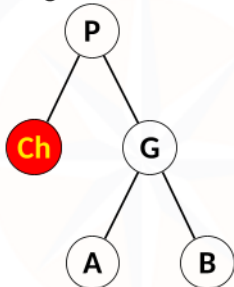
1. Ch is right child of P
2. P is root

- ▶ Perform Left Rotation on P

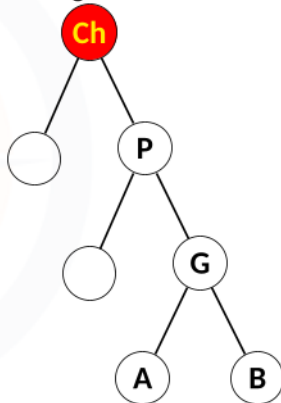
Complimentary Action of Zig Operation !!

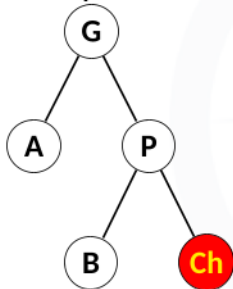


Right Rotate on G

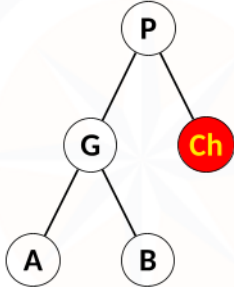


Right Rotate on P

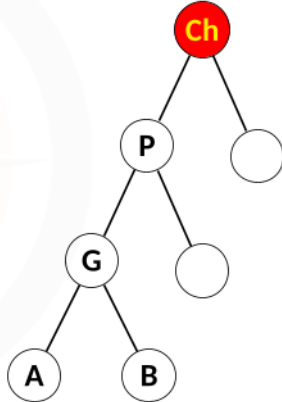




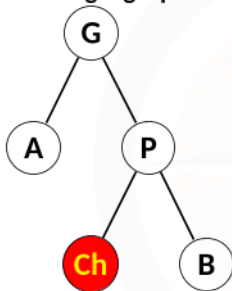
Left Rotate on G



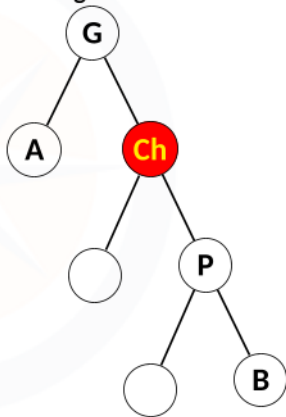
Left Rotate on P

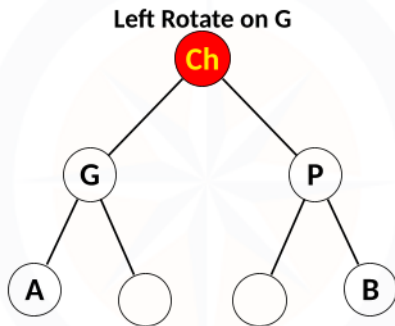


Before ZigZag Operation



Right Rotate on P

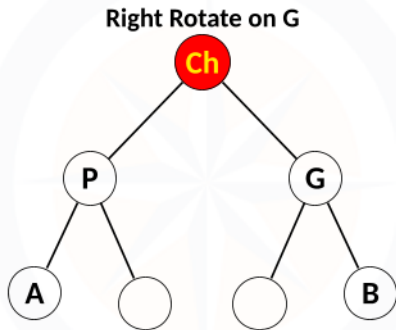






Before ZagZig Operation

Left Rotate on P





Data Structures

Splay Trees

► Note

- Algorithm is listed in the supplementary document
- Step by Step Tracing is also illustrated in the supplementary document.



Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026